Read/Write Ratio: How 100:1 Reads Shape Your Architecture

one ratio that changes almost every design decision

Watch the 3-min video

In one line: find the read-to-write ratio early; read-heavy systems (100:1) are built cache-first with replicas, CDNs and precomputed data, write-heavy systems use LSM stores, partitioning, batching and queues, and both must watch write amplification.

1 Two systems with opposite shapes

A short link is created once and then clicked a hundred times. A sensor reading is written once a second and read maybe once a day. Both store small records, yet they need almost opposite designs. The difference is one number: the **read-to-write ratio**.

The ratio compares how often data is read with how often it is written. **Read-heavy**: social feeds, product catalogs, URL shorteners, often 100:1 or more. **Write-heavy**: logging, metrics, IoT and chat ingestion. If the interviewer does not give it, ask. If they don't know, estimate it from actions per user: “each user posts 2 times a day and opens the feed 20 times, each feed load reads 50 posts” gives 2 writes vs 1,000 post reads, about 500:1.

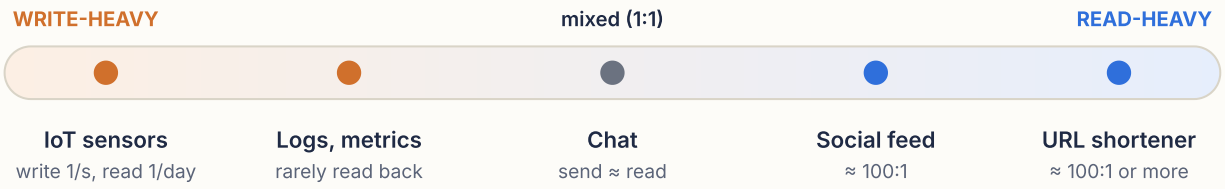


Figure 1. Where common systems sit. The same app can sit in different places for different features.

2 Read-heavy: keep reads away from the primary

The goal is to serve reads **without touching the primary database**. Four tools, usually used together:

- **Cache first.** Put Redis or Memcached in front, so most reads are memory lookups (well under a millisecond) instead of database queries.
- **Read replicas** serve what the cache misses, so the primary mostly does writes.
- **CDN** for static content: images, scripts, video, even cacheable API responses.
- **Precompute.** Denormalized views and materialized feeds do the expensive joins once, at write time, so each read is cheap.

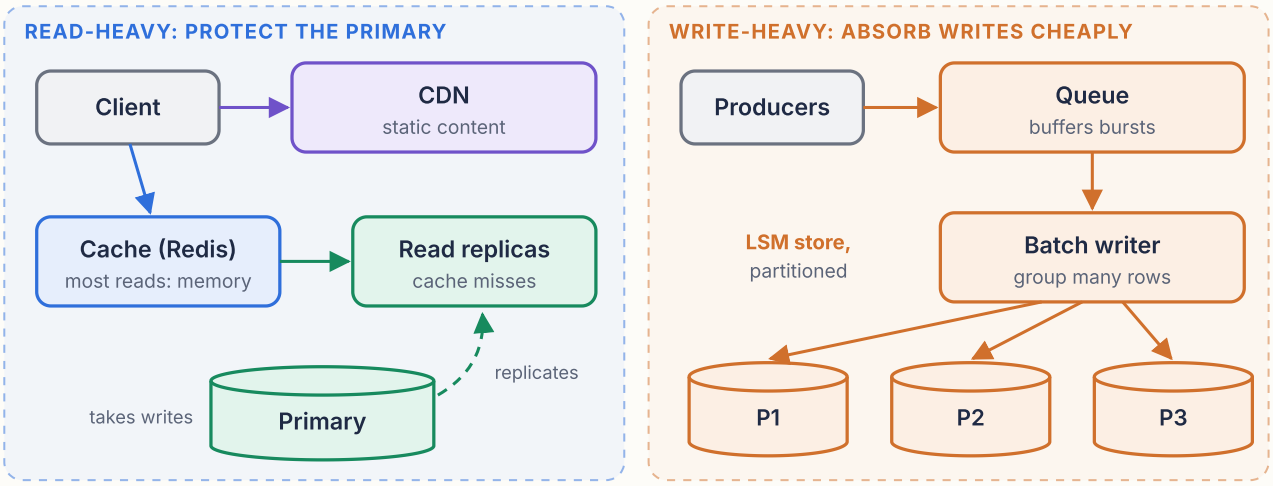


Figure 2. Left: reads hit the CDN or cache, then replicas; the primary takes the rare writes. Right: writes are queued, batched and spread over partitions of a write-optimized store.

3 Write-heavy: absorb writes cheaply

- **Write-optimized storage.** LSM-tree databases such as Cassandra (also RocksDB, ScyllaDB) append to a log and an in-memory table, then flush sorted files. They avoid updating pages in place, which is what B-tree databases do.
- **Partition the data** so writes spread across many nodes, not one primary. Pick a key that spreads load, such as sensor ID, not a timestamp.
- **Batch** writes together: one write of 500 rows beats 500 single-row writes.
- **Queue in front** (Kafka, SQS) to buffer bursts, so the store sees a smooth rate instead of spikes.

4 Watch out for write amplification

One **logical** write can become many **physical** writes:

- Every **secondary index** adds a write.
- **Replication** multiplies writes by the replication factor.
- **LSM compaction** rewrites the same data several times as files are merged.
- **Fan-out on write**, like pushing a post into a million followers' feeds, turns one write into a million.

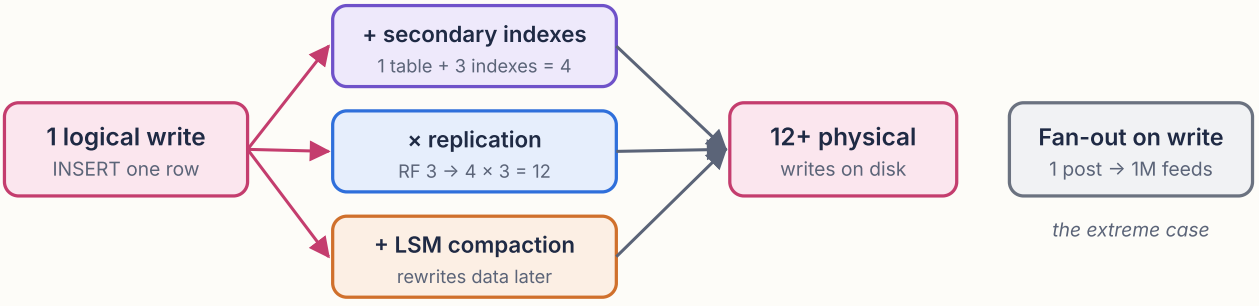


Figure 3. One insert with 3 secondary indexes and replication factor 3 is already 12 physical writes, before compaction.

So when you size a write-heavy system, size it for physical writes, not the number of API calls. Drop indexes you don't query, and think twice before fan-out on write for users with huge follower counts.

5 The URL shortener, and mixed systems

WORKED EXAMPLE: URL SHORTENER (≈100:1)

Say **100 million** new links a day: 100M ÷ 86,400 ≈ **1,200 writes/s**. At 100:1, that is about **120,000 redirects/s**.

Design **cache first**: on a redirect, look up the short code in Redis. Hot links stay in the cache, and the hit rate can reach **99%**. Only 1% misses, about 1,200 reads/s, go to the database: easy for a primary and a couple of replicas. Links never change once created, so **cache invalidation is trivial**. Writes are rare and simple: one insert per new link.

Many systems are mixed, and the ratio differs per feature. In a chat app, sending messages is write-heavy, but loading user profiles is read-heavy. So analyze the ratio **per data type** and pick the right tool for each: a cache for profiles, a write-optimized store for messages.

COMMON MISTAKES

- Never asking for the ratio, and designing one generic database for everything.
- Using one ratio for the whole app when features differ (messages vs profiles).
- Counting API writes but forgetting indexes, replication and fan-out.
- Adding read replicas first when a cache with a 99% hit rate would remove most of the load.

6 Key terms

Read/write ratio: reads per write, e.g. 100:1

Read replica: copy of the DB that serves reads

Write amplification: one logical write → many physical

Read-heavy: reads dominate; optimize the read path

Denormalization: store data pre-joined for cheap reads

Fan-out on write: copy a post into each follower's feed

Write-heavy: writes dominate; optimize ingestion

LSM tree: append-and-merge storage built for writes

Hit rate: share of reads served by the cache

7 Interview questions

Q1. How does a 100:1 read ratio influence your design? BASIC

It is read-heavy, so optimize the read path: cache first with a high hit rate, read replicas for misses, a CDN for static content and precomputed or denormalized data. Keep the rare, simple writes on the primary.

Q2. How do you design for a write-heavy workload? BASIC

Use an LSM-based store such as Cassandra, partition so writes spread over many nodes, batch writes, and put a queue in front to absorb bursts. Keep indexes minimal.

Q3. What is write amplification? BASIC

One logical write turning into many physical ones: indexes, replication, compaction, fan-out. One insert with 3 indexes at replication factor 3 is 12 physical writes.

Q4. Nobody gives you the ratio for a news feed. How do you estimate it? SCENARIO

From actions per user: say 2 posts a day and 20 feed loads of 50 posts each. That is 2 writes vs 1,000 post reads, about 500:1, so read-heavy, and a precomputed feed plus cache is worth it.

Q5. A URL shortener does 120,000 redirects/s with a 99% cache hit rate. What load reaches the database? SCENARIO

1% of 120,000 = 1,200 reads/s, plus about 1,200 inserts/s. A primary with a couple of replicas handles that. Without the cache, 120,000 reads/s would need many replicas.

Q6. A chat app: which parts are read-heavy and which write-heavy? SCENARIO

Message sending and storage are write-heavy: a partitioned LSM store keyed by conversation. Profiles, contact lists and settings are read-heavy: cache them. Choose per data type.

Q7. Why is caching a URL shortener easier than caching a product catalog? TRICKY

Short links are immutable, so a cached entry never goes stale and invalidation is trivial. Prices and stock change, so the catalog needs TTLs or invalidation on every update.

Q8. A celebrity with 50 million followers posts. Why does fan-out on write hurt here? TRICKY

One post becomes 50 million feed writes, a huge burst. Use fan-out on read for celebrities (merge their posts at read time) and fan-out on write for normal users: a hybrid.

Next: SD-D-006 · Storage Estimation backendroadmap.com